

Pressure Point

A barometric-pressure forecaster that warns people about pressure swings linked to migraines, joint pain, and low mood.

Project title	Pressure Point — Barometric Forecaster
Author	Abdul Manan
Course	Panaversity AI-101 · Agentic AI
Live app	https://pressure-point.netlify.app
AI tool used	Claude (Anthropic)
Tech stack	Single self-contained HTML file · vanilla JavaScript · SVG · Open-Meteo API

1. The idea behind the project

Many people who suffer from migraines, arthritis, or weather-sensitive joint pain notice their symptoms flare up around storms and weather changes — but few of them know why, and even fewer realise it is something they can see coming. The hidden culprit is barometric pressure: a fast drop in atmospheric pressure is a well-documented trigger for migraine attacks and joint pain.

Pressure Point turns that invisible trigger into a simple 48-hour forecast. A user types in a city (or shares their location), and the app reads the upcoming surface-pressure curve, flags any sharp swings, and translates them into plain language: when to expect a drop, and roughly how severe it will be. The goal is to give sensitive people enough warning to pre-medicate, hydrate, or lighten their schedule before a bad day arrives.

Why it counts as a real-world problem: the data already exists and is free, but it is buried in raw weather feeds that ordinary people never read. The project's value is not new data — it is translation. It takes a number most people have never heard of and tells them what it means for their own body.

2. The AI-assisted development process

The entire app was built in conversation with Claude across a small number of focused prompts, with no manual coding between iterations. The workflow followed the suggested cycle of idea → prompt → generate → test → refine.

- **Ideation.** I asked Claude for non-obvious app ideas that could be built as a single HTML file using free, open-source APIs. From the shortlist I chose the barometric-pressure concept because it solves a genuine problem and uses a data source most people overlook.

- **Specification.** Before any code was written, I had Claude draft a detailed build prompt — defining the data source, the trigger threshold, the output format, the visual design, and the constraints. Writing the spec first kept the build focused.
- **Generation.** Claude produced the complete app as one self-contained HTML file: markup, styling, and logic together, with no external libraries or build step.
- **Testing & validation.** The pressure-swing detection logic was validated against synthetic data (a flat curve, a sharp drop, and a sharp rise) to confirm it flagged the right windows and stayed quiet when nothing was happening.
- **Refinement.** After a real-world test surfaced a misleading error message, I fed the exact problem back to Claude and had it patch only that function.
- **Deployment.** The single HTML file was deployed to Netlify by drag-and-drop, giving an instantly shareable public link.

3. Prompt engineering techniques used

Several deliberate prompting techniques shaped the quality of the output:

- **Spec-first prompting.** Rather than asking for code immediately, I had the AI write a complete specification first, then build from it. This separated 'what to build' from 'how to build it' and produced a tighter result.
- **Constraint stacking.** The prompt fixed hard constraints up front — one HTML file, no dependencies, no API key, no local storage, must work on Netlify — so the model could not drift toward a heavier architecture.
- **Domain anchoring.** I supplied the specific medical threshold (a change of roughly 5 hPa over six hours) so the trigger logic reflected real research rather than a guessed value.
- **Design direction.** Instead of leaving the look to chance, the prompt named the aesthetic — a calm, medical-but-warm palette, a serif display face, a barometer-style chart — so the interface had a clear identity.
- **Targeted iteration.** For the one bug, I gave the AI the exact symptom and asked it to fix only the affected function, avoiding unnecessary rewrites of working code.

4. Iterations and improvements

Iteration 1 — Specification

Defined the data flow, the trigger threshold, the visual language, and the constraints before writing any code.

Iteration 2 — First working build

Generated the full app: city search and geolocation, Open-Meteo geocoding and forecast calls, pressure-swing detection, a hand-drawn SVG chart with shaded trigger windows, a plain-English verdict, and headline statistics — all in one file.

Iteration 3 — Honest error handling

Real-world testing exposed a misleading message (covered in detail in the next section). The geolocation handler was rewritten to report the true cause of failure instead of always blaming the user.

5. Problems faced and solutions applied

Problem 1 — The 'Use my location' button always said 'permission denied'

When the file was opened directly from disk, clicking 'Use my location' returned 'Location permission denied' even though nothing had been denied. The real cause is that browsers only allow geolocation on secure (https) pages; a file opened locally is blocked automatically, but the original code reported every failure as a denial.

Solution: the error handler was rewritten to detect the page context. It now distinguishes between an insecure (local file) page, a genuine permission block, and a timeout, and shows the correct guidance for each. Crucially, the typed-city search needs no location access at all, so the app remained fully usable while testing locally, and the location button works once deployed to Netlify over https.

Problem 2 — Verifying logic without a live network

The pressure-swing detection needed to be trustworthy before launch.

Solution: the detection function was tested against synthetic pressure series — a flat curve, a sharp drop, and a sharp rise. It correctly flagged the drop and the rise, stayed silent on the flat curve, and merged overlapping windows into a single clean band.

6. Final working features

- City search with geocoding, plus a one-tap 'Use my location' option.
- A live 48-hour surface-pressure forecast pulled from the free Open-Meteo API (no key required).
- Automatic detection of pressure swings of 5 hPa or more over six hours — the threshold commonly linked to migraine.
- A hand-drawn SVG barometer chart with trigger windows shaded in amber.
- A plain-English verdict that names the time and severity of the next likely trigger, or confirms a calm stretch ahead.
- Headline statistics: current pressure and the biggest upcoming swing.
- Clear loading and error states, and an honest medical disclaimer.
- Fully self-contained: one HTML file, no dependencies, no build step, no stored data.

7. Does the project solve a real-world problem?

Yes. Weather-triggered migraines and joint pain affect a large number of people, many of whom do not know their symptoms are forecastable. Pressure Point gives them a free, instant early-warning tool built from open data — turning a number they have never heard of into advance notice they can act on.

Disclaimer carried in the app: the tool is informational, not medical advice. Pressure sensitivity varies between individuals, and the flagged threshold reflects commonly cited research rather than a diagnosis.

Pressure Point · built with Claude · deployed on Netlify · data from Open-Meteo